

sets

CSCI
134

W O O D L E

B	E	E	C	H
A	S	P	E	N
M	A	P	L	E



so i invented this game

W O O D L E

B	E	E	C	H
A	S	P	E	N
M	A	P	L	E

building the game required
a way to check the validity
of a player's guess (is it a
five-letter type of wood?)

```
woods = ["ASPEN", "BALSA", "BEECH", "BIRCH", "CEDAR", "EBONY", "MAPLE"]
```

W O O D L E

B	E	E	C	H
A	S	P	E	N
M	A	P	L	E



extensive research went into woodle™

W O O D L E

B	E	E	C	H
A	S	P	E	N
M	A	P	L	E



my r&d team scoured the globe
for five-letter varieties of wood

each team member returned to woodle hq
with a list of the woods they'd discovered



```
member1 = ["BALSA", "BEECH", "EBONY"]
```



```
member2 = ["ASPEN", "BIRCH", "CEDAR", "EBONY", "MAPLE"]
```



```
member3 = ["BEECH", "BIRCH", "CEDAR", "MAPLE"]
```

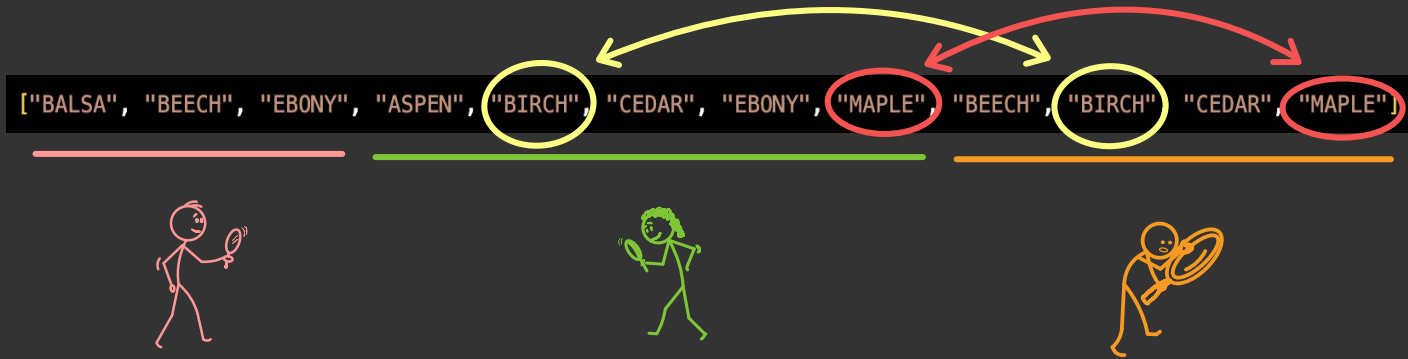
at first, we just **concatenated** these
lists into one long list of woods

```
woods = member1 + member2 + member3
```



but this produced a list with **duplicates**

```
woods = member1 + member2 + member3
```

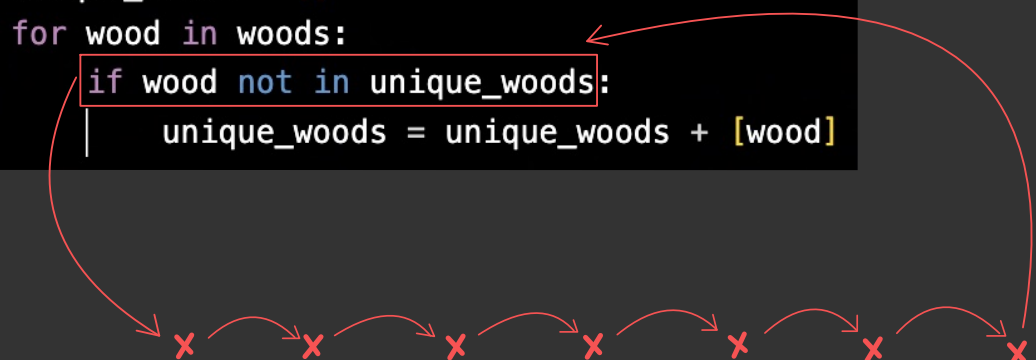


luckily we remembered
the **in** keyword

```
unique_woods = []  
for wood in woods:  
    if wood not in unique_woods:  
        | unique_woods = unique_woods + [wood]
```

but this might require checking **every** element of `unique_woods` during each iteration of the **for** statement

```
unique_woods = []  
for wood in woods:  
    if wood not in unique_woods:  
        unique_woods = unique_woods + [wood]
```



`unique_woods = ["ASPEN", "BALSA", "BEECH", "BIRCH", "CEDAR", "EBONY", "MAPLE"]`

woodle.py

```
from random import shuffle

def shuffled_nums(size):
    nums = list(range(size)) + list(range(size))
    shuffle(nums)
    return nums[:size]

def remove_duplicates(my_list):
    uniques = []
    for element in my_list:
        if element not in uniques:
            uniques = uniques + [element]
    return uniques

if __name__ == "__main__":
    remove_duplicates(shuffled_nums(20000))
```

woodle.py

```
from random import shuffle

def shuffled_nums(size):
    nums = list(range(size)) + list(range(size))
    shuffle(nums)
    return nums[:size]

def remove_duplicates(my_list):
    uniques = []
    for element in my_list:
        if element not in uniques:
            uniques = uniques + [element]
    return uniques

if __name__ == "__main__":
    remove_duplicates(shuffled_nums(80000))
```

4x longer

```
% time python woodle.py
1.72s user 0.02s system
```

```
% time python woodle.py
27.92s user 0.05s system
```

16x longer



so we went to woodle labs and
looked for a better way

eureka!



```
In [1]: set(["cedar", "maple", "balsa", "maple", "cedar"])  
Out[1]: {'balsa', 'cedar', 'maple'}
```

so we went to woodle labs and
looked for a better way

like lists, **sets** are a way to
group together expressions

{ expression , expression , expression , expression , ... , expression }

```
In [1]: {'balsa', 'cedar', 'maple'}
```

```
Out[1]: {'balsa', 'cedar', 'maple'}
```

```
In [2]: {'balsa'}
```

```
Out[2]: {'balsa'}
```

```
In [3]: {}
```

```
Out[3]: {}
```

```
In [4]: {'balsa', 'cedar', 'maple', 'balsa'}
```

```
Out[4]: {'balsa', 'cedar', 'maple'}
```

```
In [5]: {'balsa', 'cedar', 'cedar', 'maple', 'balsa'}
```

```
Out[5]: {'balsa', 'cedar', 'maple'}
```

like lists, **sets** are a way to
group together expressions

{ expression , expression , expression , expression , ... , expression }

↑
curly
braces

```
In [1]: {'balsa', 'cedar', 'maple'}  
Out[1]: {'balsa', 'cedar', 'maple'}
```

```
In [2]: {'balsa'}  
Out[2]: {'balsa'}
```

```
In [3]: {}  
Out[3]: {}
```

```
In [4]: {'balsa', 'cedar', 'maple', 'balsa'}  
Out[4]: {'balsa', 'cedar', 'maple'}
```

```
In [5]: {'balsa', 'cedar', 'cedar', 'maple', 'balsa'}  
Out[5]: {'balsa', 'cedar', 'maple'}
```

↑
curly
braces

empty sets are permitted

{ expression , expression , expression , expression , ... , expression }

```
In [1]: {'balsa', 'cedar', 'maple'}  
Out[1]: {'balsa', 'cedar', 'maple'}
```

```
In [2]: {'balsa'}  
Out[2]: {'balsa'}
```



```
In [3]: {}  
Out[3]: {}
```

```
In [4]: {'balsa', 'cedar', 'maple', 'balsa'}  
Out[4]: {'balsa', 'cedar', 'maple'}
```

```
In [5]: {'balsa', 'cedar', 'cedar', 'maple', 'balsa'}  
Out[5]: {'balsa', 'cedar', 'maple'}
```

sets cannot contain duplicate elements

{ expression , expression , expression , expression , ... , expression }

```
In [1]: {'balsa', 'cedar', 'maple'}  
Out[1]: {'balsa', 'cedar', 'maple'}
```

```
In [2]: {'balsa'}  
Out[2]: {'balsa'}
```

```
In [3]: {}  
Out[3]: {}
```



```
In [4]: {'balsa', 'cedar', 'maple', 'balsa'}  
Out[4]: {'balsa', 'cedar', 'maple'}
```



```
In [5]: {'balsa', 'cedar', 'cedar', 'maple', 'balsa'}  
Out[5]: {'balsa', 'cedar', 'maple'}
```

sets are **not ordered**: there is no such thing as element 0 or element 1 of a set

{ expression , expression , expression , expression , ... , expression }

```
In [1]: woods = {"ASPEN", "BALSA", "BEECH"}
```

```
In [2]: woods[0]
```

```
-----  
TypeError
```

```
Cell In[2], line 1
```

```
----> 1 woods[0]
```

```
TypeError: 'set' object is not subscriptable
```

```
In [3]: len(woods)
```

```
Out[3]: 3
```

but we can ask for the **number** of elements

{ expression , expression , expression , expression , ... , expression }

```
In [1]: woods = {"ASPEN", "BALSA", "BEECH"}
```

```
In [2]: woods[0]
```

```
-----  
TypeError
```

```
Cell In[2], line 1
```

```
----> 1 woods[0]
```

```
TypeError: 'set' object is not subscriptable
```

```
In [3]: len(woods)
```

```
Out[3]: 3
```

we can **iterate** over the elements of a set using a **for** statement

```
{ expression , expression , expression , expression , ... , expression }
```

```
In [1]: woods = {"ASPEN", "BALSA", "BEECH"}
```

```
In [2]: for wood in woods:  
...:     print(wood)
```

```
ASPEN  
BALSA  
BEECH
```


we can **convert** sets to lists
and lists to sets

{ expression , expression , expression , expression , ... , expression }

```
In [1]: woods = {"ASPEN", "BALSA", "BEECH"}
```

```
In [2]: wood_list = list(woods)
```

```
In [3]: wood_list
```

```
Out[3]: ['ASPEN', 'BALSA', 'BEECH']
```

```
In [4]: wood_set = set(wood_list)
```

```
In [5]: wood_set
```

```
Out[5]: {'ASPEN', 'BALSA', 'BEECH'}
```

when we convert a list to a set,
its **duplicates** are automatically **removed**

{ expression , expression , expression , expression , ... , expression }

```
In [1]: wood_list = ["ASPEN", "BALSA", "BALSA", "CEDAR", "ASPEN"]
```

```
In [2]: wood_set = set(wood_list)
```

```
In [3]: wood_set
```

```
Out[3]: {'ASPEN', 'BALSA', 'CEDAR'}
```

woodle.py

```
from random import shuffle

def shuffled_nums(size):
    nums = list(range(size)) + list(range(size))
    shuffle(nums)
    return nums[:size]
```

```
def remove_duplicates(my_list):
    uniques = []
    for element in my_list:
        if element not in uniques:
            uniques = uniques + [element]
    return uniques
```

```
if __name__ == "__main__":
    remove_duplicates(shuffled_nums(80000))
```

woodle.py

```
from random import shuffle

def shuffled_nums(size):
    nums = list(range(size)) + list(range(size))
    shuffle(nums)
    return nums[:size]
```

```
def remove_duplicates(my_list):
    return list(set(my_list))
```

```
if __name__ == "__main__":
    remove_duplicates(shuffled_nums(80000))
```



```
% time python woodle.py
27.92s user 0.05s system
```

```
% time python woodle.py
1.71s user 0.03s system
```

other things about sets

sets support the
in keyword

two sets are
equal if they
have the same
elements

```
In [1]: woods = {"BALSA", "CEDAR", "ASPEN"}
```

```
In [2]: "CEDAR" in woods
```

```
Out[2]: True
```

```
In [3]: "BARCH" in woods
```

```
Out[3]: False
```

```
In [4]: {'a', 'b', 'c'} == {'b', 'a', 'c'}
```

```
Out[4]: True
```

```
In [5]: {'a', 'b', 'c'} == {'b', 'd', 'c'}
```

```
Out[5]: False
```

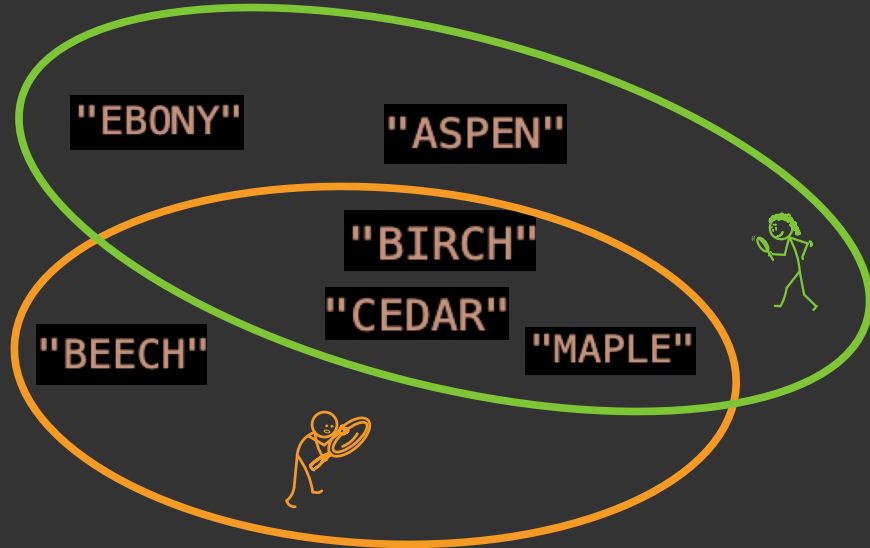
how do we
merge
two sets?



```
member2 = {"ASPEN", "BIRCH", "CEDAR", "EBONY", "MAPLE"}
```



```
member3 = {"BEECH", "BIRCH", "CEDAR", "MAPLE"}
```



not this
way



```
member2 = {"ASPEN", "BIRCH", "CEDAR", "EBONY", "MAPLE"}
```



```
member3 = {"BEECH", "BIRCH", "CEDAR", "MAPLE"}
```

```
In [3]: member2 + member3
```

```
TypeError
```

```
Traceback (most recent call last)
```

```
Cell In[3], line 1
```

```
----> 1 member2 + member3
```

```
TypeError: unsupported operand type(s) for +: 'set' and 'set'
```





```
member2 = {"ASPEN", "BIRCH", "CEDAR", "EBONY", "MAPLE"}
```



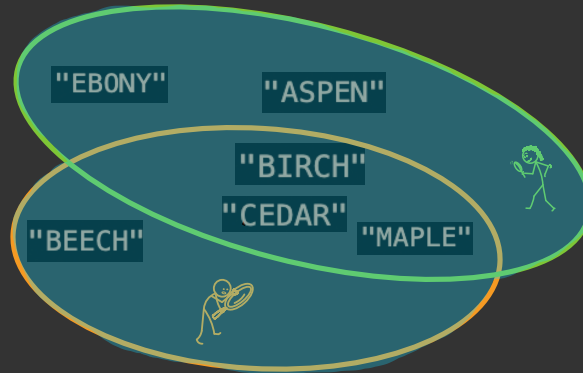
```
member3 = {"BEECH", "BIRCH", "CEDAR", "MAPLE"}
```

union



```
In [4]: member2 | member3  
Out[4]: {'ASPEN', 'BEECH', 'BIRCH', 'CEDAR', 'EBONY', 'MAPLE'}
```

```
In [5]: member2 & member3  
Out[5]: {'BIRCH', 'CEDAR', 'MAPLE'}
```





```
member2 = {"ASPEN", "BIRCH", "CEDAR", "EBONY", "MAPLE"}
```

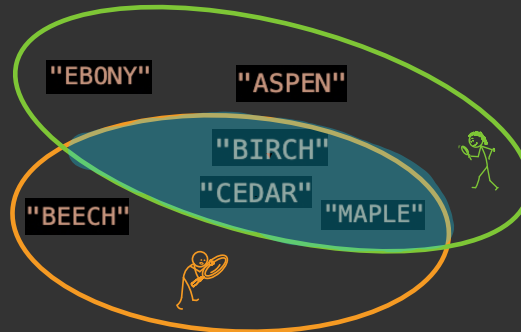


```
member3 = {"BEECH", "BIRCH", "CEDAR", "MAPLE"}
```

intersection →

```
In [4]: member2 | member3  
Out[4]: {'ASPEN', 'BEECH', 'BIRCH', 'CEDAR', 'EBONY', 'MAPLE'}
```

```
In [5]: member2 & member3  
Out[5]: {'BIRCH', 'CEDAR', 'MAPLE'}
```



expressions so far

type	example
int	7
float	3.14
str	"williams"
bool	3 > 4
function	abs
NoneType	print("hi")
range	range(1,5)
list	["mh", "rb", "jb", "rb"]
dict	{"a": 1, "b": 2, "c": 3}
set	{"mh", "rb", "jb"}